

inferencia de tipos en java/var(java10)

➡ Simplificando el código desde Java 10

ACHRAF FALEH

¿Qué es la Inferencia de Tipos?

- *Es la capacidad del compilador de Java para deducir automáticamente el tipo de dato de una variable basándose en el valor asignado.*
- *Java sigue siendo un lenguaje de tipado estático (el tipo no cambia tras la asignación).*
- ***Objetivo:** Reducir la verbosidad (código repetitivo) sin perder seguridad de tipos.*

Intoduction

La inferencia de tipos es un mecanismo mediante el cual el compilador puede deducir el tipo de una variable a partir del valor que se le asigna durante su inicialización. En Java, esta característica se implementa a través de la palabra clave `var`, que indica al compilador que debe determinar el tipo de la variable automáticamente.

Antes de Java 10, siempre teníamos que declarar explícitamente el tipo de nuestras variables:

```
1 String mensaje = "Hola mundo";  
2 ArrayList<String> nombres = new ArrayList<>();  
3 HashMap<Integer, String> mapa = new HashMap<>();
```

Con la introducción de `var`, ahora podemos escribir:

```
1 var mensaje = "Hola mundo"; // El compilador infiere que es de tipo String
2 var nombres = new ArrayList<String>(); // El compilador infiere que es ArrayList<String>
3 var mapa = new HashMap<Integer, String>(); // El compilador infiere que es HashMap<Integer, String>
```

Reglas y Limitaciones (Crucial)

- *Variables locales solamente: Solo funciona dentro de métodos, bloques (for), o bloques (if).*
- *Inicialización obligatoria: La variable debe inicializarse en la misma línea de la declaración para que el compilador sepa el tipo.*
- *No para null: No puedes usar var si inicializas con null, porque el compilador no puede deducir el tipo .*
- *No para campos de clase: No se puede usar en variables de instancia (campos) o parámetros de métodos.*

Uso adecuado de var

→ Cuándo usar var

1. Cuando el tipo es obvio por el contexto:

```
1 var contador = 0; // Obviamente es un int
2 var nombre = "Juan"; // Obviamente es un String
```

2. Con constructores de objetos genéricos donde el tipo aparece dos veces:

```
1 // Antes de var
2 Map<String, List<Integer>> mapaComplicado = new HashMap<String, List<Integer>>();
3
4 // Con var
5 var mapaComplicado = new HashMap<String, List<Integer>>();
```

3. En bucles for-each:

```
1 // Antes de var
2 for (Map.Entry<String, List<Integer>> entrada : mapaComplicado.entrySet()) {
3     // código
4 }
5
6 // Con var
7 for (var entrada : mapaComplicado.entrySet()) {
8     // código
9 }
```

4. Con variables intermedias en expresiones complejas:

```
1 var resultado = servicio.procesarDatos()
2     .filtrar(Predicado.porFecha())
3     .ordenar(Comparador.porNombre())
4     .limitar(10);
```

→ Cuándo evitar var

1. Cuando el tipo no es obvio por el contexto:

```
1 // Evitar esto
2 var resultado = calcular(); // ¿Qué tipo devuelve calcular()?
```

2. Con literales numéricos cuando el tipo específico es importante:

```
1 // Mejor evitar
2 var numero = 5; // ¿Es un int, un long, un byte?
3
4 // Mejor ser explícito si necesitamos un tipo específico
5 long valorGrande = 5L;
```

3. Con tipos genéricos sin especificar:

```
1 // Evitar esto
2 var lista = new ArrayList<>(); // ¿Lista de qué?
3
4 // Mejor así
5 var listaDeNumeros = new ArrayList<Integer>();
```


Resumen

La inferencia de tipos con `var` en Java representa un paso importante en la evolución del lenguaje para reducir la verbosidad y mejorar la legibilidad del código. Si bien Java sigue siendo un lenguaje fuertemente tipado, la inferencia de tipos nos permite escribir código más conciso sin sacrificar la seguridad de tipos. La clave para utilizar `var` correctamente es asegurarnos de que el tipo sea fácilmente deducible del contexto, lo que mantiene la claridad y la mantenibilidad del código. Cuando se utiliza adecuadamente, `var` puede hacer que nuestro código sea más limpio y más fácil de leer, especialmente en situaciones donde la declaración de tipos explícitos resulta redundante o excesivamente verbose.